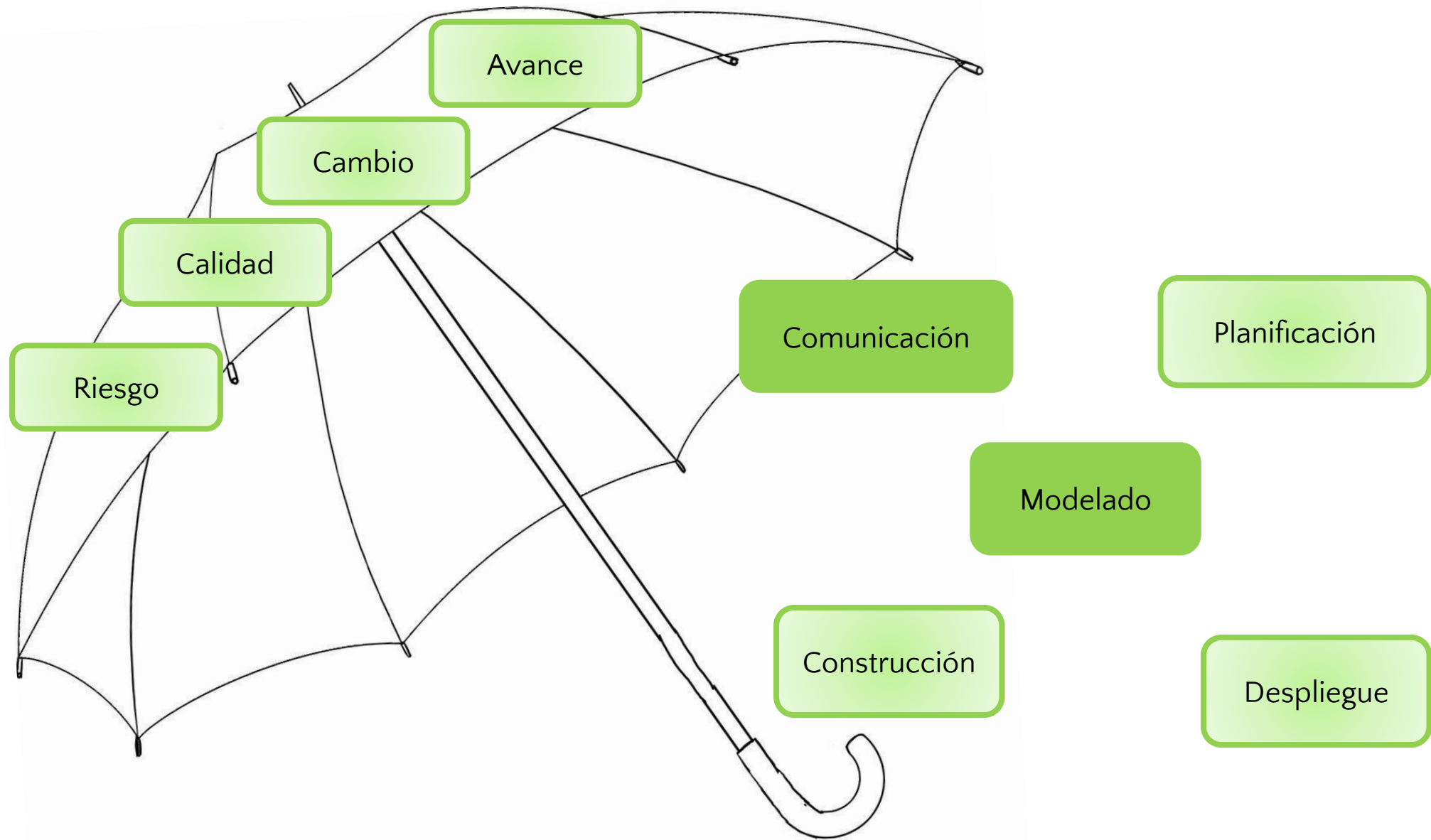


Introducción a la ingeniería de software





1

Diseño

Definiciones. Características.

Diseñar un sistema es determinar un conjunto de componentes y de interfaces entre componentes que satisfagan un conjunto específico de requerimientos.

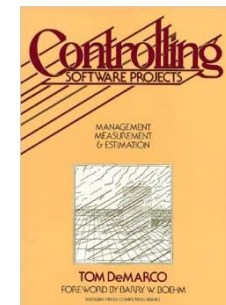
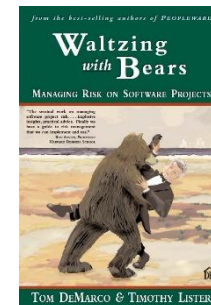
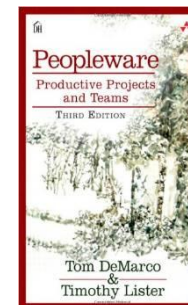
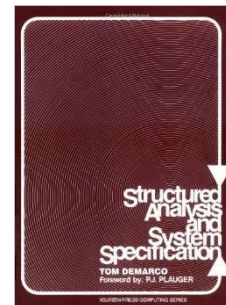
DeMarco (1982)

“



Tom DeMarco

- (1940)
- Análisis y Diseño Estructurados
- Administración de Proyectos de Software





Descomposición

- Toda descomposición separa el diseño en partes llamadas **módulos** o **componentes**.
- Un sistema es **modular** cuando una de las actividades la realiza un único componente y que además tiene bien definidas todas sus entradas y salidas.



Descomposición

- Descomposición modular: descripciones de alto nivel de las funciones de cada componente.
- Descomposición orientada por datos: basada en las estructuras externas de los datos.
- Descomposición orientada por eventos: basado en los eventos a manejar en el sistema. Cómo los eventos cambian el estado del sistema.



Descomposición: diseño de sistema de gestión académica

Descomposición modular (funcional)

Organizado a partir de procesos o funcionalidades. Cada módulo actúa sobre varios datos. Módulos:

- de Gestión de alumnos
- de Gestión de materias
- de Gestión de profesores
- de inscripciones
- de Gestión de reportes

Descomposición orientada por datos

Organizado a partir de los datos (entidades) involucrados. Cada módulo es responsable del estado y operaciones sobre sus propios datos.

Módulos:

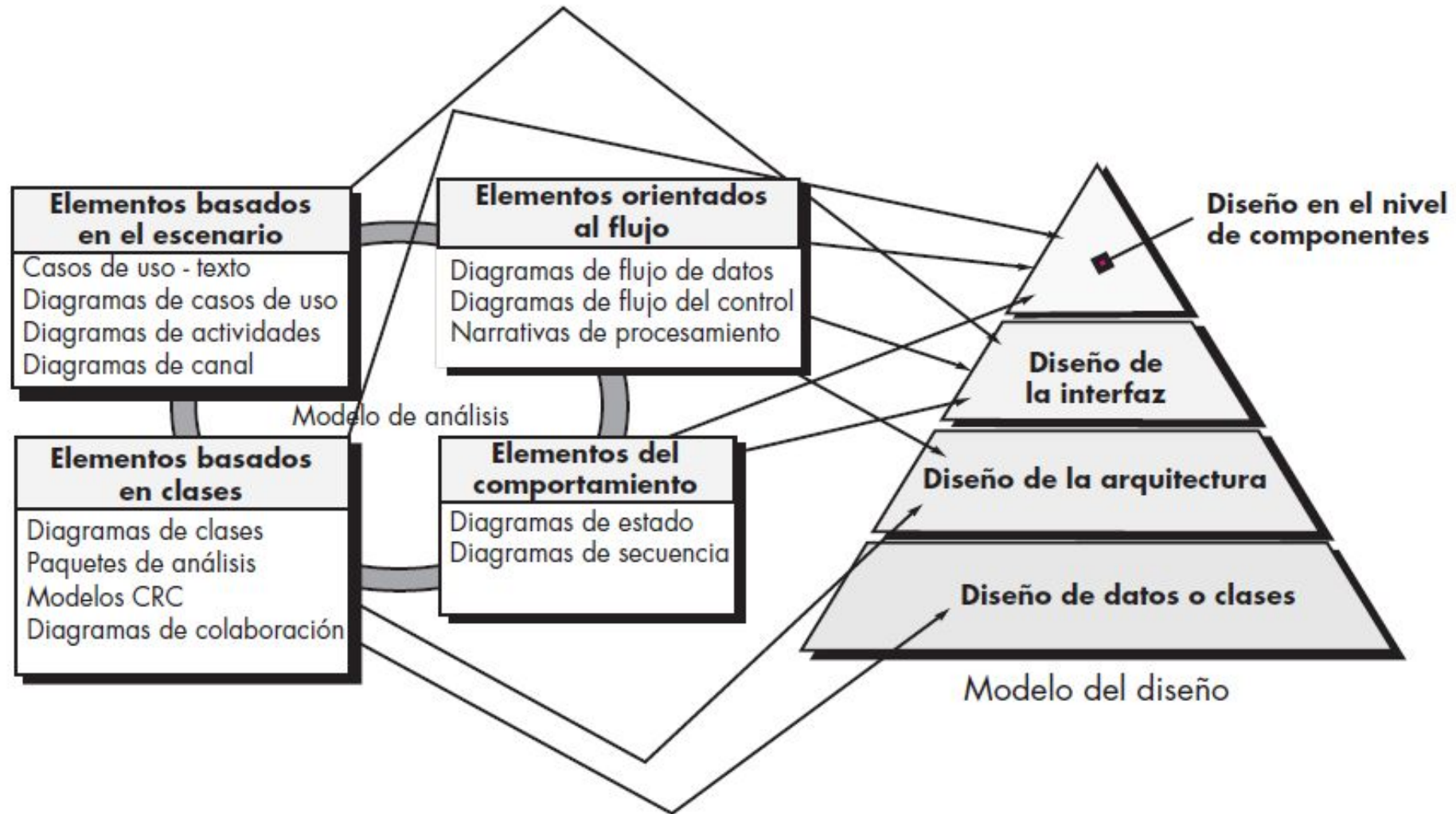
- Alumno
- Materia
- Profesor
- Inscripción

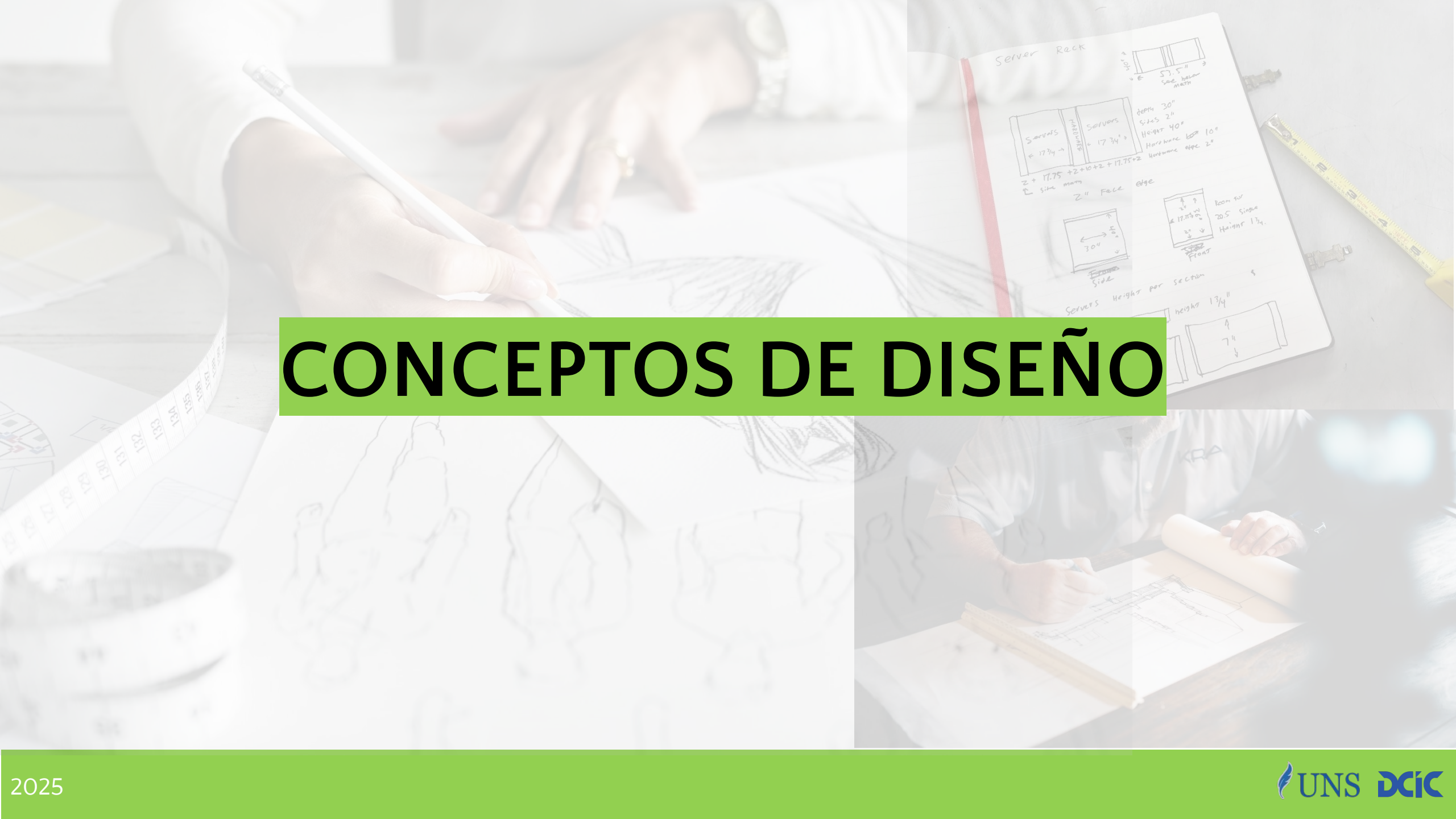


Descomposición

- Diseño “de afuera hacia adentro”: enfoque de *caja negra*. Basado en las entradas del usuario.
- Diseño orientado a objetos: identificar clases de objetos y sus interrelaciones. A nivel alto: descripción de objetos. A niveles bajos: atributos de los objetos, acciones.

Obteniendo el modelo de diseño





CONCEPTOS DE DISEÑO



Conceptos de diseño

Abstracción

Se plantean distintos niveles de abstracción.



Abstracciones de

- **procedimiento:** secuencia de instrucciones
- **datos:** conjunto que describe a un objeto



Conceptos de diseño

Arquitectura

Estructura de organización de los componentes, la forma en la que estos interactúan, las estructuras de datos que utilizan.

A partir de ella se realizará el diseño más detallado.

Se hace uso de patrones.



Conceptos de diseño

Patrones

Describe una estructura de diseño que resuelve un problema particular de diseño dentro de un contexto específico.

Veremos algunos patrones más adelante en esta clase.



Conceptos de diseño

Separación de intereses

Cualquier problema complejo se resuelve más fácilmente si se lo subdivide en partes que pueden resolverse de manera independiente.

Tiene efecto directo en el diseño y en muchos otros aspectos.

La complejidad disminuye si los problemas son más simples



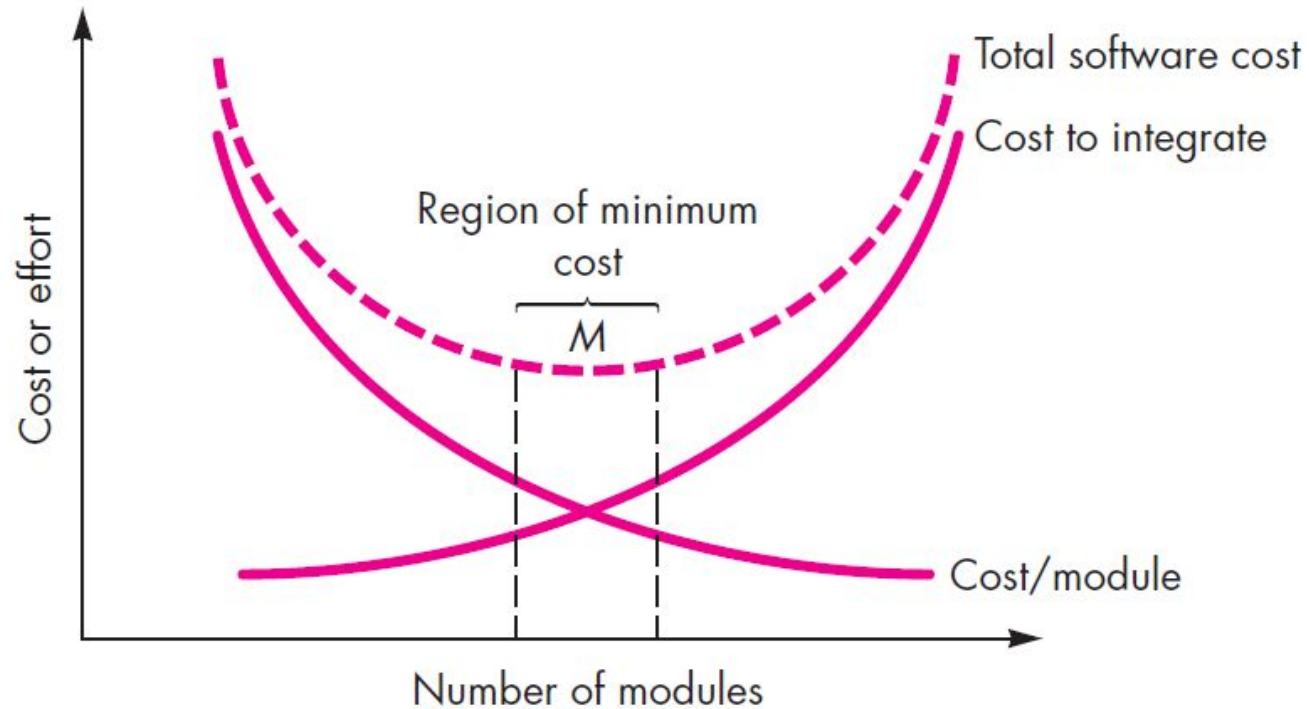
Conceptos de diseño

Modularidad

- Toda descomposición separa el diseño en partes llamadas **módulos** o **componentes**.
- Un sistema es **modular** cuando una de las actividades la realiza un único componente y que además tiene bien definidas todas sus entradas y salidas.

Conceptos de diseño

- Modularidad (Descomposición modular)
- Componentes independientes pero integrados





Conceptos de diseño

Ocultamiento de la información

Principio: la información de cada módulo debe ser inaccesible a los que no necesiten de ella.

La modularidad se consigue mediante módulos que solo intercambien la información necesaria para lograr que el software funcione.

Ventajas: al realizar pruebas, al hacer modificaciones y al dar mantenimiento



Conceptos de diseño

Independencia funcional

Cada módulo debe resolver un conjunto específico de requerimientos.

Resultado directo de: división de problemas y ocultamiento de información.



Conceptos de diseño

Refinamiento

Proceso evolutivo de elaboración.

Es el proceso por medio del cual se disminuye el nivel de abstracción.

2

Diseño arquitectónico



Diseño arquitectónico





Niveles de diseño

Diseño de arquitectura

Asocia capacidades del sistema con componentes que las implementarán. Módulos y sus interconexiones.

Diseño de código

Algoritmos y estructuras de datos más primitivas del lenguaje de programación.

Diseño ejecutable

Nivel de detalle aún más bajo

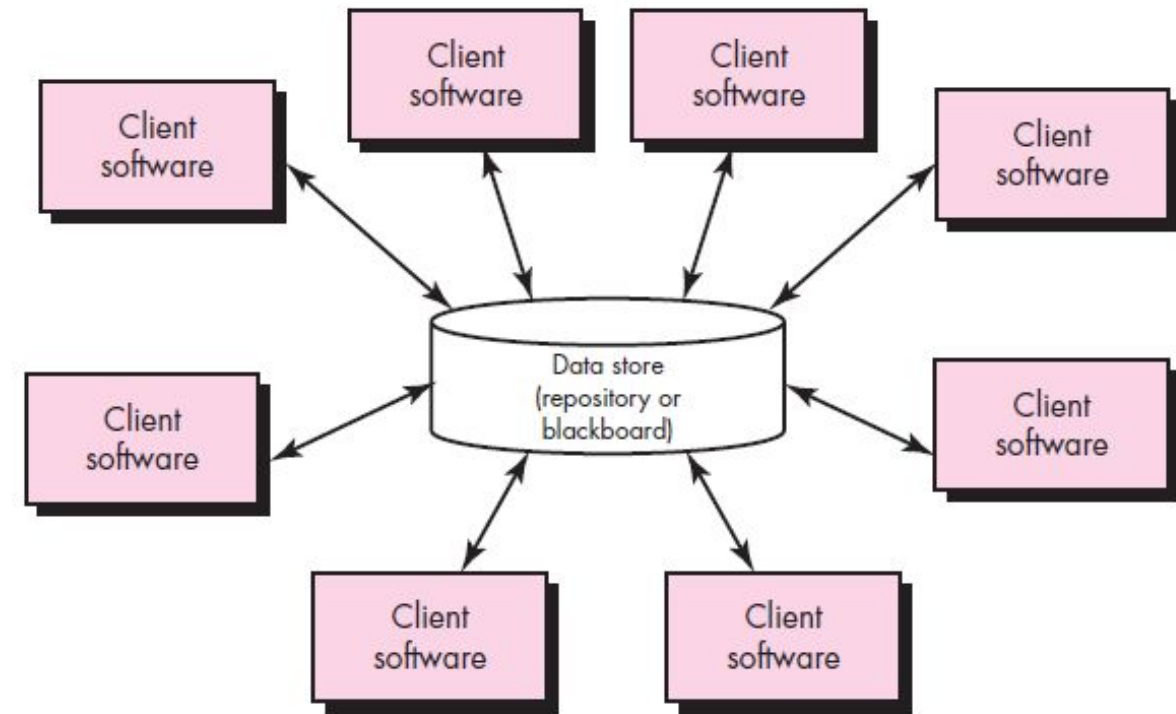


Estilos de arquitectura

- ◉ Centradas en los datos (Repositorios)
- ◉ De flujos de datos (Tuberías y filtros)
- ◉ De llamada y retorno
- ◉ Diseño orientado a objetos
- ◉ Capas

Centrada en datos: repositorios

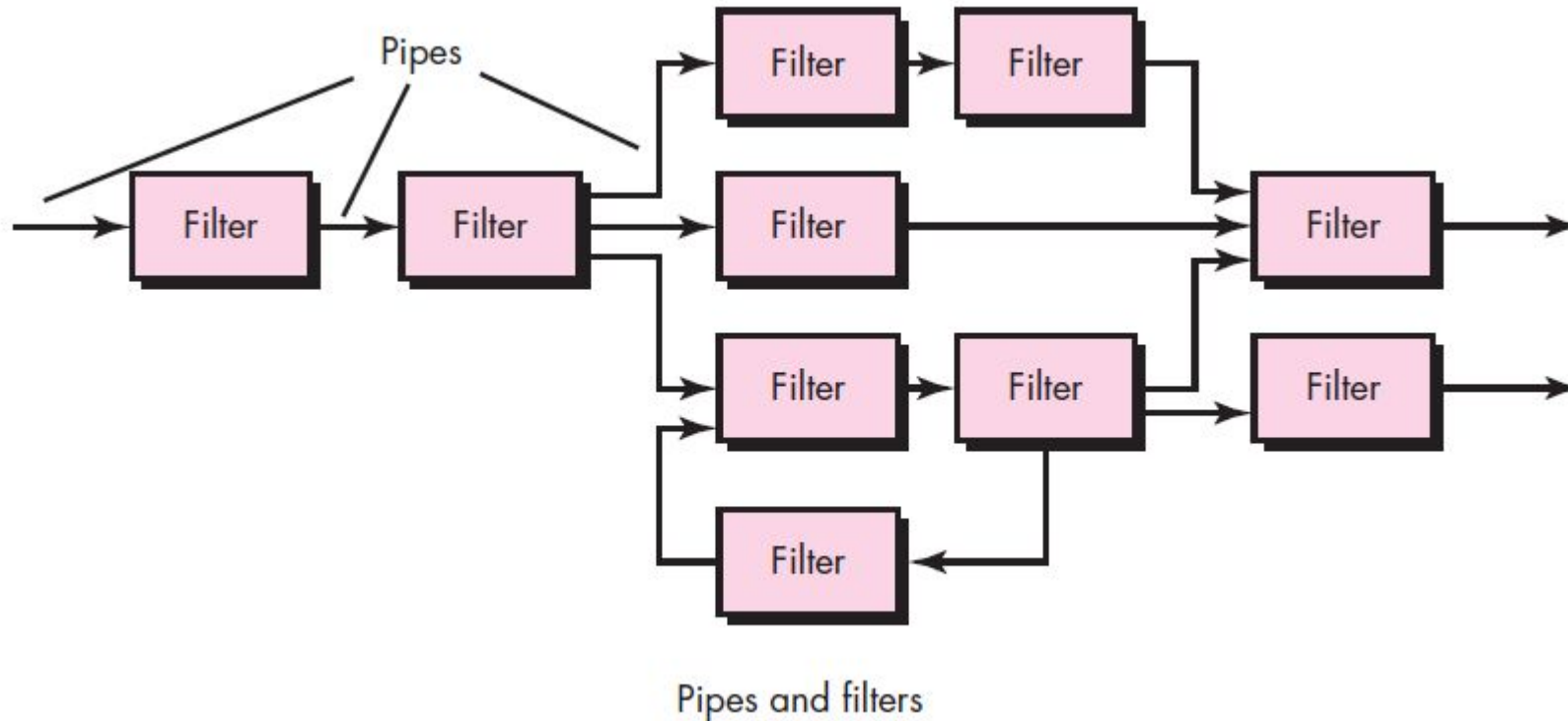
- Almacenamiento central de datos
- Conjunto de componentes que operan sobre el almacenamiento: almacenar, recuperar, actualizar
- Ventaja: disponibilidad de los datos
- Desventaja: el formato debe ser aceptable para todos los clientes que lo utilizan





Flujo de datos: Tuberías y filtros

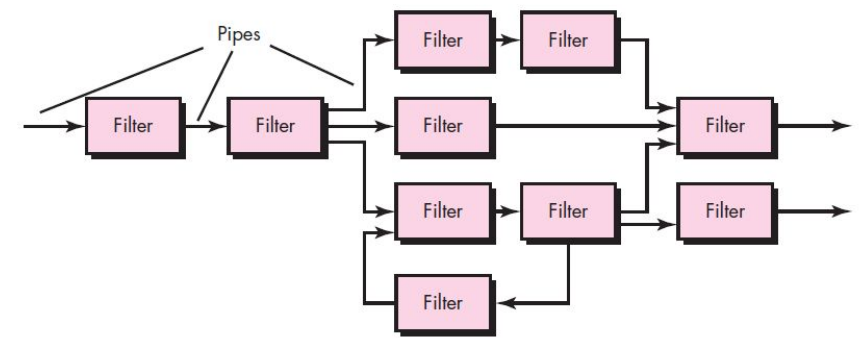
- Tuberías: corrientes de datos para entrada y salida
- Filtros: realizan las transformaciones de los datos





Flujo de datos: Tuberías y filtros

- Los filtros son independientes entre sí
- Propiedades útiles
- Comprender los efectos del sistema sobre la entrada y la salida y la composición de las transformaciones
- Se reutilizan fácilmente los filtros en otros sistemas
- La evolución es más simple: incorporar o quitar filtros
- Facilita la simulación del sistema para analizar propiedades
- Ejecución concurrente

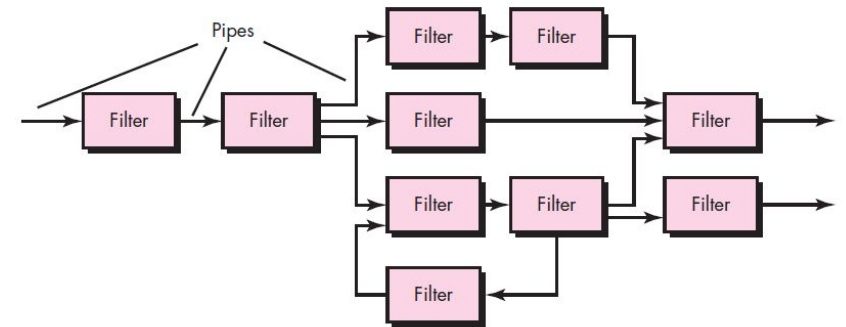


Pipes and filters



Flujo de datos: Tuberías y filtros

- Desventajas
 - Alienta el procesamiento por lotes, no es apto para aplicaciones interactivas
 - Mantener la correspondencia entre dos corrientes de datos relacionadas
 - Al ser independientes los filtros pueden duplicar funcionalidad que efectúan otros



Pipes and filters



De llamada y retorno

- Clásica estructura jerárquica
 - Un programa principal
 - Varios subprogramas componentes



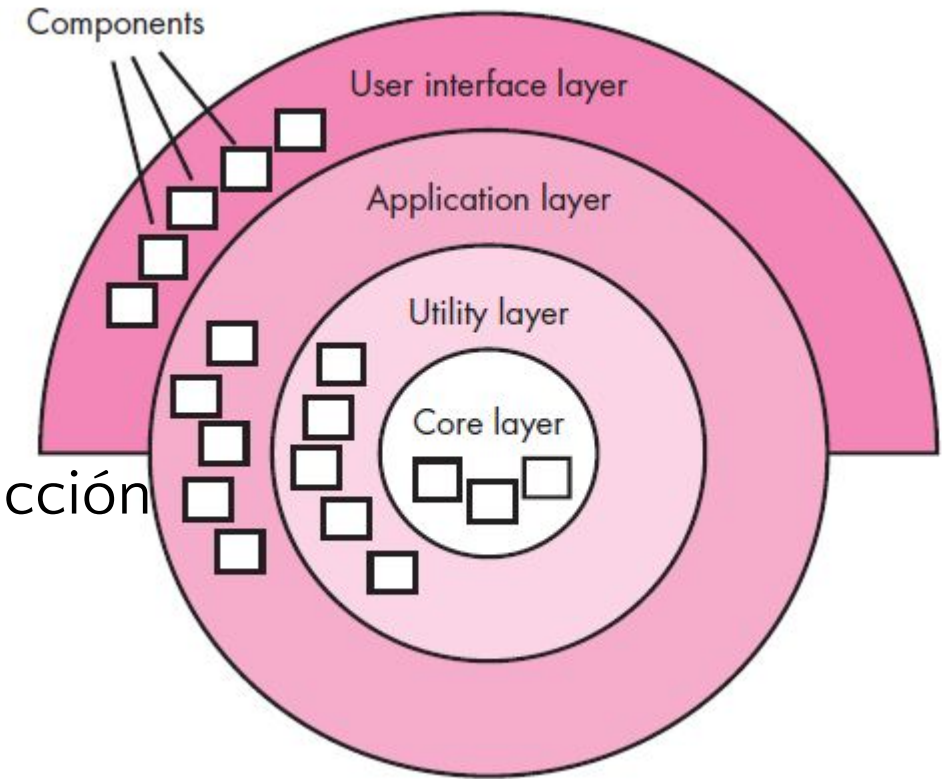
Diseño orientado a objetos

- Es un enfoque de desarrollo de software que organiza tanto el problema como su solución como una colección de objetos.
- En la representación incluye tanto los datos (y su estructura) como el comportamiento.
- Esto hace que en muchos casos la descripción del problema, y el diseño de la solución sea muy similar o casi idéntica.
- La representación puede ocultarse a los restantes objetos del sistema (encapsulación)



Capas

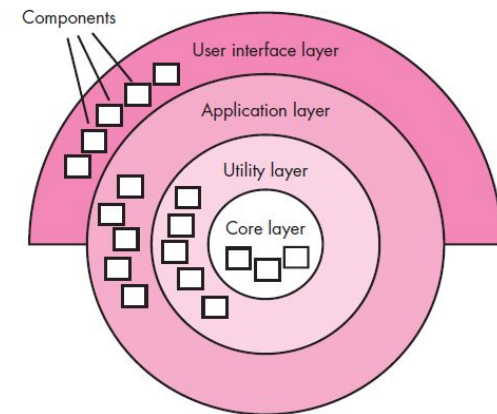
- Los modelos estratificados tienen organización jerárquica
- Cada capa:
 - Presta servicios a la inmediata superior
 - Es cliente de la inferior
- El diseño incluye los protocolos de interacción





Capas

- Niveles superiores tienen mayor abstracción
- Ventaja: agregar o modificar capas es sencillo ya que el cambio solo afecta a las capas adyacentes
- Desventajas
 - No siempre es fácil estructurar sistemas de esta manera
 - Se agrega el costo de la coordinación de las capas





Diseño de interfaz

REGLAS DORADAS – Theo Mandel

- ◉ Dejar el control al usuario
- ◉ Reducir la carga de memoria del usuario
- ◉ Hacer que la interfaz sea consistente

Cada grupo contiene un conjunto de reglas (y palabras clave)



Diseño de interfaz

Dejar el control al usuario. Principios de diseño

- ◉ No obligar a realizar acciones innecesarias o no deseadas
- ◉ Dar interacción flexible (teclado o mouse)
- ◉ Interacción interrumpible y reversible
- ◉ Permitir personalización: facilitar aumento de habilidad
- ◉ Ocultar tecnicismos internos
- ◉ Permitir manipulación directa de objetos en pantalla



Diseño de interfaz

Reducir la carga de memoria del usuario. Principios de diseño

- ◉ Reducir demanda de memoria a corto plazo
- ◉ Que lo preestablecido sea significativo
- ◉ Atajos intuitivos
- ◉ Metáforas en la distribución
- ◉ Revelar información de manera progresiva



Diseño de interfaz

Hacer que la interfaz sea consistente. Principios de diseño

- Colocar la tarea en contexto significativo
- Consistencia en toda la familia de aplicaciones
- No hacer cambios sobre las expectativas del usuario

3

Buenas prácticas

O más conceptos para un buen diseño



Características de un buen diseño

- Se intenta que cada componente del diseño sea lo más independiente del resto posible
- Mejora la comprensión de cada componente individual
- Es más sencillo de modificar una característica de un componente si no afecta a los demás
- Es más fácil identificar fallas
- Para medir la independencia de los componentes usamos dos conceptos
- Acoplamiento
- Cohesión



Acoplamiento

- Dos componentes están altamente acoplados cuando existe mucha dependencia entre ellos.
- Los componentes poco acoplados tienen pocas dependencias y/o las interconexiones son débiles.

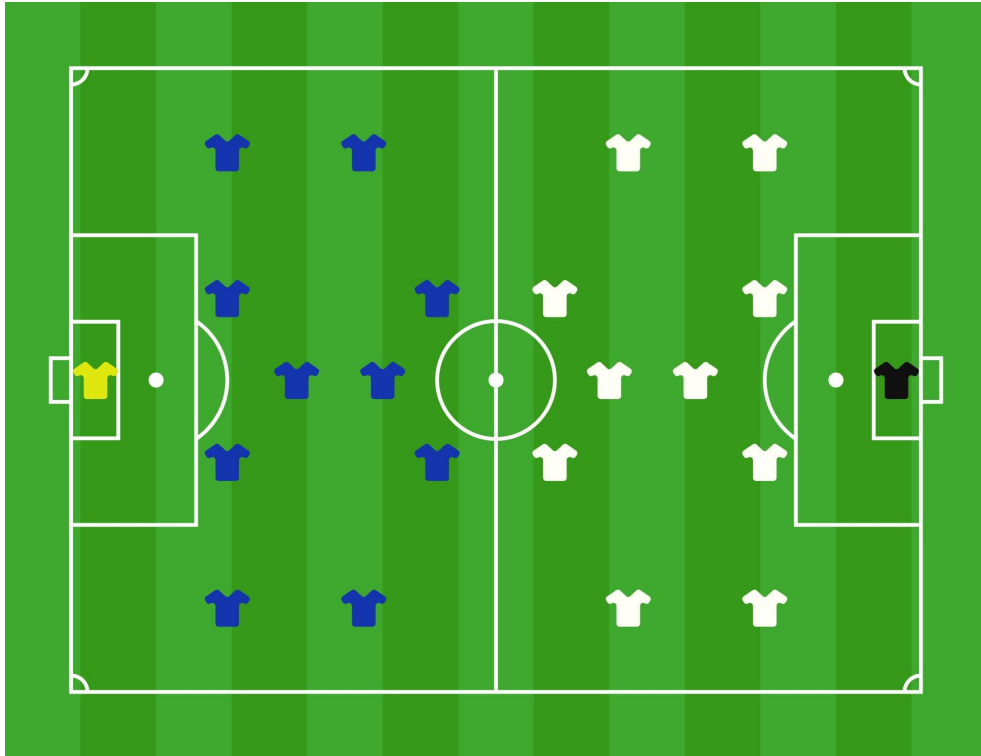


Acoplamiento

- El acoplamiento depende de
 - Las referencias hechas de un componente a otro
 - La cantidad de datos pasados entre componentes
 - El grado de control de un componente sobre otro
 - La complejidad de la interfaz entre los componentes



Acoplamiento en un equipo



↑ Un delantero solo recibe con éxito los pases de determinado mediocampista

↑ Un arquero solo siente seguridad para atajar estando al arco en la cancha de su club

↓ Ante la lesión de un jugador, cualquiera de los suplentes puede ingresar y el equipo completo puede seguir jugando sin sufrir el cambio



Acoplamiento

- Hay distintos tipos de acoplamientos, unos menos convenientes que otros.
- Cuando un componente A modifica a un componente B, este es completamente dependiente del que lo modifica. Se denomina **acoplamiento de contenido**.
- A modifica el valor de un item de B
- A ramifica su ejecución en B



Acoplamiento

- Cuando un almacenamiento de datos es común a dos o más módulos se tiene **acoplamiento común**.
- Cuando un componente pasa parámetros de control a otro se dice que hay **acoplamiento de control**.
- Cuando se utiliza una estructura de datos compleja para pasar información de un módulo a otro se dice que hay **acoplamiento de molde**.
- Si solo comparten datos es **acoplamiento de datos**.



Acoplamiento

- de contenido
 - común
- de control
- de molde
- por datos
- no acoplado

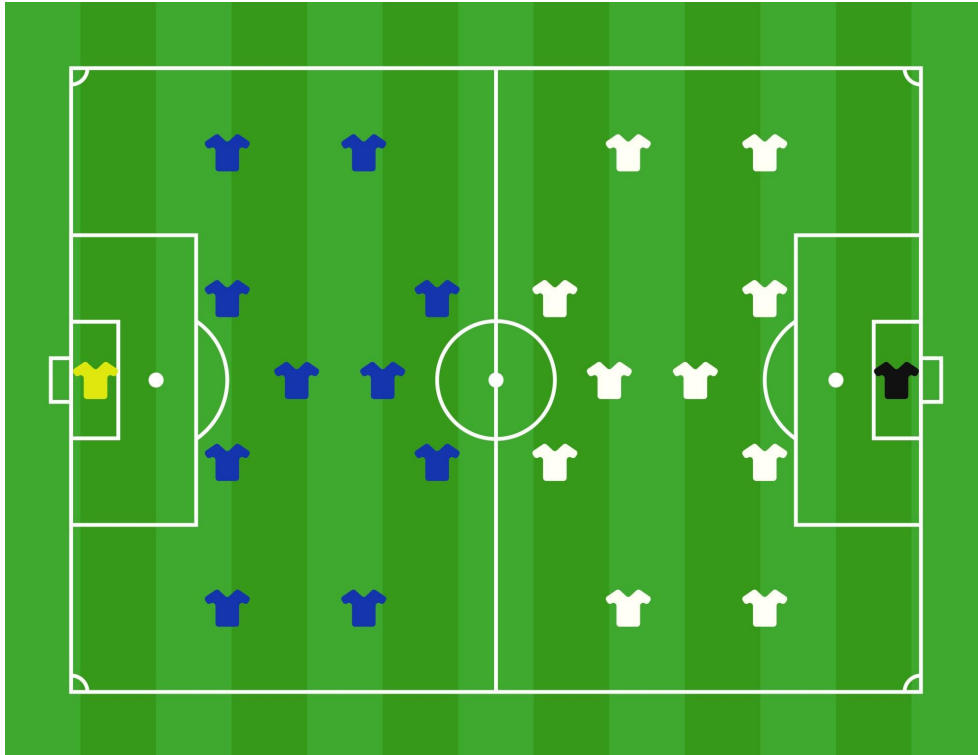


Cohesión

- La cohesión se refiere al grado de adhesión (unión) interna que tiene el componente.
- A mayor grado de cohesión, más relacionadas entre sí están sus partes internas
- Un componente es cohesivo si todos sus elementos están orientados a realizar una única tarea y son esenciales para llevarla a cabo



Cohesión en un equipo



↑ El equipo tiene una meta única y clara (ej: ganar un partido), todos los jugadores la conocen y se enfocan en ella

↑ Cada jugador juega un rol específico para servir a la táctica elegida para ganar el partido

↓ Cada jugador persigue un objetivo distinto, no juega de acuerdo a un rol propio



Cohesión

- La **cohesión coincidental** ocurre cuando las partes no tienen relación alguna entre sí.
- En la **cohesión lógica** los elementos están relacionados lógicamente. Ej: todas las funciones sirven para la entrada de datos, sin importar el origen.
- Cuando las funciones de un componente están relacionadas por el momento en el que ocurren o son invocadas, se trata de **cohesión temporal**.



Cohesión

- En ocasiones las funciones se agrupan únicamente porque están relacionadas por un procedimiento, hay un orden de ejecución. En este caso se dice que el componente tiene **cohesión de procedimiento**.
- Cuando un componente asocia funciones que trabajan (operan, producen) en el mismo conjunto de datos, tiene **cohesión comunicativa**.



Cohesión

- Si la relación entre las funciones es que una produce la salida que sirve de entrada a la siguiente, la **cohesión** es **secuencial**.
- La cohesión ideal es la **cohesión funcional**. En estos componentes las partes están relacionadas por la función que realizan: es una única función y todas las partes son esenciales para realizarla.



Cohesión

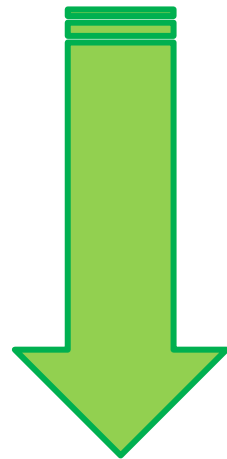
- Funcional
- Secuencial
- Comunicacional
- Procedimental
- Temporal
 - Lógica
- Coincidental



Buenos diseños

- Un buen diseño modular minimiza el acoplamiento y maximiza la cohesión

Acoplamiento



Cohesión



4

Diseño de situaciones no deseadas

O más conceptos para un buen diseño



Diseño de situaciones no deseadas

- La especificación de requerimientos dice lo que debe hacer el sistema. No explicita lo que se supone que no va a ocurrir.
- ¿Qué pasa si ocurre algo no planificado?
- Identificar lo que conocemos como **excepciones**: situaciones que se sabe que no son las esperadas.
- El diseño puede incluir **manejo de excepciones** de modo que el sistema pueda reaccionar de manera satisfactoria ante un evento de este tipo.



Excepciones

- Lanzamiento Ariane 5 (501)





Excepciones

- Incidente Ariane 5 (501)
- Reuso de código de Ariane 4
- Motor más rápido y potente genera números más grandes
- Se produce “overflow” y falla de la computadora de vuelo seguido de falla de la computadora principal.
- La falla generalizada le da demasiada potencia y se desintegra en menos de 40 segundos.
- ~370 millones de USD en pérdidas



Excepciones

- Lanzamiento Ariane 5 (501)

Video: https://youtu.be/gp_D8r-2hww

Otro video - explicación (en inglés):
<https://youtu.be/5tJPXYA0Nec>





Excepciones

- Algunas comunes:
- Fracasar al proporcionar un servicio
- Proporcionar un servicio o datos erróneos
- Corrupción de datos

- Acciones a tomar
- Reintentar: restaurar el sistema y probar otra forma
- Corregir: restaurar el sistema y probar nuevamente
- Informar: restaurar el sistema e informar de lo ocurrido sin proporcionar el servicio



Diseño de situaciones no deseadas

- Errores humanos introducen defectos en el software.
- Si el defecto es en el diseño se puede propagar a los demás productos del proceso.
- **Falla**: desvío del sistema de su comportamiento requerido. Es percibida por el usuario.
- El **defecto** origina la falla. El defecto solo lo puede percibir el desarrollador.
- No todo defecto deriva en falla: las condiciones pueden no darse nunca.



Técnicas para mejorar el diseño

- Reducción de la complejidad
- Reducir la complejidad desde que se detecta en los modelos.
- Diseño por contrato
- Considera el sistema como un conjunto de componentes que se comunican.
- Las comunicaciones están especificadas en contratos que deben definir exactamente lo que hace cada uno y cómo interactúan.
- No pueden garantizar exactitud pero dan base para pruebas y validaciones



Resumen

Conceptos de diseño

- Abstracción
- Arquitectura
- Patrones
- Separación de intereses
- Modularidad
- Ocultamiento de información
- Independencia funcional
- Refinamiento

Diseño arquitectónico

- Niveles de diseño
- Estilos arquitectónicos

Buenas prácticas de diseño

- Acoplamiento
- Cohesión
- Manejo de excepciones



Bibliografía



- *Ingeniería del software. Un enfoque práctico* – R. Pressman
 - Conceptos de diseño. (Capítulo 8 / Capítulo 12)
 - Diseño de la arquitectura (Capítulo 9 / Capítulo 13)
 - Diseño en el nivel de componentes (Capítulo 10 / Capítulo 14)

- Theo Mandel – *Reglas de oro del diseño de la interfaz de usuario*
<https://theomandel.com/resources/golden-rules-of-user-interface-design/>
<https://theomandel.com/wp-content/uploads/2012/07/Mandel-GoldenRules.pdf>

Template: www.slidescarnival.com

Mg. M. Clara Casalini. – Dr. Axel Soto – 2025

Introducción a la ingeniería de Software – Ingeniería en Sistemas de Información

Departamento de Ciencias e Ingeniería de la Computación – Universidad Nacional del Sur